Formal Soundness Proof for Athena

Anonymous Artifact

1 Scope, Claims, and Trusted Computing Base

This appendix gives a soundness proof for Athena. All semantics, translations, and proof obligations are stated as definitions and lemmas. The proof is not mechanized.

What we prove. Let Athena return one of $\{\textsc{Terminating}, \textsc{NonTerminating}, \textsc{Unknown}\}$ on an input C program P. We prove:

- If Athena returns Terminating, then P has *no* infinite execution in the reference semantics.
- If Athena returns NonTerminating, then P has an *infinite* execution in the reference semantics.
- If Athena returns Unknown, we make *no* claim.

We do not claim completeness.

Trusted components (TCB). Our end-to-end theorem is conditional only on:

1. correctness of the reference small-step semantics for the supported C fragment;
2. correctness of the C-to-LLVM compilation for the supported fragment and chosen integer mode (stated precisely as Assumption 8);
3. SMT solver soundness for all validity checks performed by Athena (Assumption 11);
4. the standard small-step operational semantics of LLVM for the supported instruction subset.

All other properties (rewriting allocator freshness/disjointness, invariant soundness from checked inductiveness, and LLVM→LTS correctness for the supported subset) are proved below.

2 Source Language and Reference Semantics

2.1 C Fragment

Definition 1 (Types)**.** *Types τ range over machine integers* $\texttt{i}w$*, pointers $\tau^\star$, arrays $\tau[n]$, and structs* $\texttt{struct}\{f_i : \tau_i\}$*.*

Definition 2 (Values)**.** *Values v are machine integers (or bitvectors in BV mode), pointers, and aggregates. A pointer is a pair (b, δ) with block id b and offset $\delta \in \mathbb{Z}$.*

Definition 3 (Memory (block-based))**.** *A memory μ is block-based: each allocation returns a fresh block b with size* $\mathrm{size}(b) \in \mathbb{Z}_{>0}$*. Loads/stores are defined only when in-bounds and aligned according to the reference layout rules.*

Definition 4 (Source State)**.** *A source state σ is a triple (μ, ρ, ℓ) where: μ is memory, ρ maps variables to values, and ℓ is a control location.*

Definition 5 (Small-step Semantics)**.** *Let P be a fixed source C program. We write $\langle P, \sigma\rangle \longrightarrow_C \langle P, \sigma'\rangle$ for one C step under a CompCert-style operational semantics for the supported fragment.*

Definition 6 (Well-definedness Predicate)**.** $\mathsf{WF}(P, \sigma)$ *holds iff the next source step from $\langle P, \sigma\rangle$ is defined: all semantic side-conditions for the imminent operation hold (in-bounds and aligned loads/stores; defined pointer arithmetic; defined casts/comparisons; etc.).*

Assumption 1 (End-to-End Scope)**.** *Out of scope end-to-end: explicit* `free`*,* `union`*, concurrency/atomics/volatile, and general recursion.* ATHENA *may rewrite tail recursion into equivalent loops; if unsupported constructs occur,* ATHENA *returns* UNKNOWN*.*

Assumption 2 (Pointer Conversions/Comparisons)**.** *Pointer-to-int and int-to-pointer casts are allowed only in restricted cases modeled by rewriting (e.g., byte-wise access within the same allocation). Pointer comparisons are only those defined by the reference semantics (e.g., equality with null or within the same object when defined).*

Assumption 3 (Defined Executions for Soundness Statements)**.** *All executions quantified by Theorem 5 satisfy $\mathsf{WF}(P, \sigma)$ at every step.*

Definition 7 (Live Blocks)**.** *A source block b is* live *in state $\sigma = (\mu, \rho, \ell)$, written* $\mathrm{live}(b, \sigma)$*, iff b is allocated in μ and (i) reachable from the environment ρ via values and aggregates (including pointers in memory), or (ii) is a global block. Since* `free` *is out of scope, liveness is monotone: if* $\mathrm{live}(b, \sigma)$ *and $\sigma \longrightarrow_C^* \sigma'$, then* $\mathrm{live}(b, \sigma')$*.*

3 Pointer-to-Array Rewriting

3.1 May-Points-to Information and Block Construction

Definition 8 (Pointer Abstraction)**.** *Let $\mathcal{P}$ be the set of pointer variables (including pointer-typed fields). Let Q be the set of objects that may be pointed to (globals, address-taken locals, arrays, and allocation sites). For each pointer expression $p \in \mathcal{P}$, a may-points-to analysis yields a set $Q(p) \subseteq Q$.*

Assumption 4 (Soundness of May-Points-to Analysis)**.** *For every execution and runtime pointer value computed for a pointer expression p, if p evaluates to the address of an object o at runtime, then $o \in Q(p)$.*

Definition 9 (Memory Blocks)**.** *A block-construction algorithm computes a finite set of blocks $\mathcal{M} = \{\langle m_i, \sigma_i\rangle \mapsto (\mathcal{P}_i, T_i, \mathcal{Q}_i)\}$, where each block m_i stores the contents of all objects in $\mathcal{Q}_i$. The mapping is conservative: multiple distinct source objects may be stored in the same block.*

Definition 10 (Block Map $M(\cdot)$)**.** *$M(x) = m_i$ means variable/object x is represented in block m_i after rewriting; $M(x) =$* `null` *means x remains a direct scalar.*

Lemma 1 (Routing Adequacy)**.** *Let p evaluate at runtime to the address of an object o. Under Assumption 4, $o \in Q(p)$ and thus $o \in \mathcal{Q}_i$ for some block m_i. Therefore rewriting routes dereferences of p through a block that stores o.*

Proof. By Assumption 4, for any execution in which pointer expression p evaluates to the address of object o, we have $o \in Q(p)$. By construction of the block partition, every object in $Q(p)$ is assigned to some block m_i with $o \in \mathcal{Q}_i$. The rewriting routes dereferences of p to the block determined by $Q(p)$, and hence to a block that contains o. □

3.2 Target Memory Model with Constructive Allocation

Definition 11 (Athena Block Store)**.** *Each block id i has: (i) an array m_i (byte-addressed or typed, as defined in Section 4.2 of the paper), (ii) a natural-number next pointer* $\mathrm{next}_i \in \mathbb{Z}_{\geq 0}$*, and (iii) a set* $\mathrm{Alloc}_i \subseteq \mathbb{Z} \times \mathbb{Z}$ *of allocated intervals.*

Definition 12 (Alignment Rounding)**.** *Let* $\mathrm{align}(\tau) \in \mathbb{Z}_{>0}$ *be the required alignment for type τ. Define* $\mathrm{round}_a(x) = \min\{y \geq x \mid y \equiv 0 \pmod{a}\}$.

Definition 13 (Concrete Allocation Primitive)**.** $\texttt{alloc}_{m_i,\tau}(n)$ *returns base*

$$\alpha = \mathrm{round}_{\mathrm{align}(\tau)}(\mathrm{next}_i), \quad \mathrm{next}_i := \alpha + n, \quad \mathrm{Alloc}_i := \mathrm{Alloc}_i \cup \{(\alpha, \alpha + n)\}.$$

Lemma 2 (Allocation Freshness, Disjointness, and Alignment)**.** *In any execution, intervals added to* Alloc_i *by repeated calls to* $\texttt{alloc}_{m_i,\tau}(\cdot)$ *are: (i) pairwise disjoint; (ii) strictly increasing by base address; and (iii) aligned to* $\mathrm{align}(\tau)$.

Proof. By Definition 13, each allocation chooses $\alpha \geq \mathrm{next}_i$ and aligned by rounding, then updates next_i to $\alpha + n$. Any future allocation rounds a value $\geq \alpha + n$, yielding a base $\alpha' \geq \alpha + n$. Thus intervals are disjoint and bases strictly increase. Alignment follows by construction. □

Definition 14 (Typed/Byte Loads and Stores)**.** *For a block m_i and type τ,* $\texttt{load}_{m_i,\tau}(\alpha)$ *and* $\texttt{store}_{m_i,\tau}(\alpha, v)$ *encode/decode values at address α. Typed blocks may be element-indexed; byte blocks are byte-indexed.*

Assumption 5 (Faithful Data Layout and Encoding)**.** *Type sizes, alignments, field offsets, and byte encodings in* `load`/`store` *match the reference semantics on the chosen target architecture.*

3.3 Constructive Base Map

Definition 15 (Base Map Construction)**.** *For each* source allocation event *that creates a fresh block b of size n and dynamic type τ, the rewriting performs exactly one call* $\texttt{alloc}_{m_{\beta(b)},\tau}(n)$ *and records its return value as $base(b)$. This defines base as a partial function on live blocks, extended monotonically as new blocks are allocated (Definition 7 and no-*`free` *scope).*

Lemma 3 (Base Map Injectivity on Live Blocks)**.** *At any state, if $b_1 \neq b_2$ are live source blocks mapped to the same target block id $i = \beta(b_1) = \beta(b_2)$, then their intervals $[base(b_1), base(b_1) + \mathrm{size}(b_1))$ and $[base(b_2), base(b_2) + \mathrm{size}(b_2))$ are disjoint in m_i.*

Proof. By Definition 15, each $base(b)$ is returned by a distinct call to `alloc`. Disjointness follows from Lemma 2. □

3.4 Rewriting Function

Definition 16 (Rewriting)**.** *Let P be in scope.* $\mathsf{Rewrite}(P)$ *applies the syntax-directed rules R1–R15 defined in Section 4.2.4 (Figure 5) of the paper, guided by $M(\cdot)$ and points-to sets, and inserts m_i and helper primitives* $\texttt{alloc}_{m_i,\tau}$, $\texttt{load}_{m_i,\tau}$, $\texttt{store}_{m_i,\tau}$.

3.5 State Relation and Definedness Preservation

Definition 17 (State Correspondence $\mathcal{R}_{CA}$)**.** *$\mathcal{R}_{CA}(\sigma, \sigma_A)$ holds between source $\sigma = (\mu, \rho, \ell)$ and rewritten σ_A iff:*

1. *For every live source block b, $\beta(b)$ is defined and $base(b)$ is defined as in Definition 15.*
2. ***Pointer representation:** if $\rho(x) = (b, \delta)$ then x's rewritten representation stores $base(b)+\delta$; null is represented by 0.*
3. ***Allocation correspondence:** for each live b, the interval $[base(b), base(b) + \mathrm{size}(b))$ is included in $\mathrm{Alloc}_{\beta(b)}$, and disjointness holds by Lemma 3.*
4. ***Content preservation:** defined source loads/stores at (b, δ) of type τ match the rewritten `load`/`store` at $base(b) + \delta$ in $m_{\beta(b)}$.*
5. ***Defined pointer equality:** any defined source pointer-equality test corresponds to integer equality of rewritten addresses.*
6. ***Control alignment:** control locations correspond under rewriting.*

Lemma 4 (Definedness Preservation from Source to Rewriting)**.** *Assume $\mathcal{R}_{CA}(\sigma, \sigma_A)$ and $\mathsf{WF}(P, \sigma)$ at corresponding control points. Then the next rewritten control-point segment executes only defined helper operations and reaches the next target control point.*

Proof. Since $\mathsf{WF}(P, \sigma)$ holds, all side-conditions needed for the impending source semantic rule hold. Helper computations consist of integer arithmetic and calls to `alloc`/`load`/`store`. By Lemma 2 allocations are fresh and aligned; by Assumption 5, load/store encodings respect alignment and bounds corresponding to the defined source access. Assumption 2 excludes unsupported casts/comparisons. Thus the segment is defined and terminates. □

Lemma 5 (Single-Step Forward Simulation of Rewriting)**.** *If $\mathcal{R}_{CA}(\sigma, \sigma_A)$ and $\mathsf{WF}(P, \sigma)$ and $\langle P, \sigma\rangle \longrightarrow_C \langle P, \sigma'\rangle$, then there exist σ'_A and $k \geq 1$ such that $\langle \mathsf{Rewrite}(P), \sigma_A\rangle \longrightarrow_A^* \langle \mathsf{Rewrite}(P), \sigma'_A\rangle$ and $\mathcal{R}_{CA}(\sigma', \sigma'_A)$.*

Proof. By induction on the source step derivation, using Lemma 4 to justify helper definedness. Memory cases use Lemma 3 and Assumption 5. Structural cases are homomorphic. □

Definition 18 (Control Points)**.** *Source/target control points are the boundaries between source-level constructs, excluding internal helper straight-line computations introduced by rewriting.*

Lemma 6 (Bounded Helper Progress)**.** *There exists a constant B (depending only on P) such that between two consecutive visits to target control points, at most B rewritten small-steps occur.*

Proof. Each rewriting rule expands into a finite straight-line fragment; helper primitives contain no loops. Take the maximum expansion size across all rule instances in P. □

Lemma 7 (Backward Simulation at Control Points)**.** *Assume $\mathcal{R}_{CA}(\sigma, \sigma_A)$ at corresponding control points and the target reaches the next control point. Then there exists a source step $\sigma \longrightarrow_C \sigma'$ such that $\mathcal{R}_{CA}(\sigma', \sigma'_A)$ holds at the next point.*

Proof. By case analysis on the rewriting rule instance executed between the control points. Since the target segment is the image of exactly one source semantic rule application, and the source was well-defined, we reconstruct the source step and use Assumption 5 to align offset computations. □

Theorem 1 (Infinite-Run Preservation and Reflection of Rewriting)**.** *Let $P_A = \mathsf{Rewrite}(P)$ and assume $\mathcal{R}_{CA}(\sigma_0, \sigma_{A0})$ initially and Assumption 3. Then:*

1. *If P has an infinite execution from σ_0, then P_A has an infinite execution from σ_{A0}.*
2. *If P_A has an infinite execution from σ_{A0}, then P has an infinite execution from σ_0.*

Proof. (1) Iterate Lemma 5. Each source step yields at least one target step.

(2) Let π_A be an infinite execution of P_A. By Lemma 6, π_A visits target control points infinitely often. Apply Lemma 7 to each control-point segment to reconstruct infinitely many source steps, yielding an infinite execution of P. □

4 Integer Semantics Modes

Assumption 6 (Mode Semantics)**.** ATHENA *operates in exactly one of the following integer semantics modes:*

- ***Bit-Vector****: integers have fixed width w and all operations follow $\mathsf{BV}(w)$ semantics.*
- ***Modulo****: the source program is instrumented so arithmetic matches wraparound machine-integer semantics.*
- ***Unbounded****: integers range over $\mathbb{Z}$; soundness requires that no overflow occurs.*

Lemma 8 (Bit-Vector Operator Soundness)**.** *In Bit-Vector mode, each LTS-level bit operation denotes exactly the corresponding $\mathsf{BV}(w)$ operation of the LLVM integer semantics.*

Proof. Each arithmetic and bitwise operation in the LTS is defined to apply the corresponding fixed-width $\mathsf{BV}(w)$ operator to its operands. The encoding preserves operand values and applies the same truncation, extension, and overflow behavior as LLVM's bit-vector semantics. Thus, the result of each LTS operation coincides with the LLVM semantics of the corresponding instruction. □

Lemma 9 (Modulo Instrumentation Soundness)**.** *In Modulo mode, the inserted wraparound instrumentation makes each arithmetic operation observationally equivalent to the reference machine-integer semantics.*

Proof. In Modulo mode, each arithmetic operation is instrumented to compute results modulo 2^w, where w is the target machine word size. This matches the wraparound behavior of machine integers in the reference semantics. Since the instrumentation preserves control flow and memory effects, each instrumented operation is observationally equivalent to its source-level counterpart. □

Assumption 7 (Overflow-Freedom for Unbounded Mode)**.** *When* ATHENA *runs in Unbounded mode, integer overflow does not occur on any execution in the reference semantics.*

5 C-to-LLVM and LLVM-to-LTS Translation

5.1 LLVM Semantics and Compilation

Definition 19 (LLVM State and Semantics)**.** *Let Σ_{LLVM} be LLVM states (PC, SSA registers, and an aggregate memory model). We write $\langle \mathsf{Comp}(P_A), \Sigma\rangle \longrightarrow_{\mathrm{LLVM}} \langle \mathsf{Comp}(P_A), \Sigma'\rangle$.*

Assumption 8 (Compiler Correctness for Infinite Executions)**.** *For the supported C fragment and selected integer mode, compilation preserves and reflects infinite executions: P_A has an infinite execution iff $\mathsf{Comp}(P_A)$ has an infinite LLVM execution.*

5.2 LTS Semantics

Definition 20 (Transition System)**.** *A transition system T is $(S, s_0, \rightarrow)$. We write $s \longrightarrow_T s'$ and* $\mathsf{Reach}(T) = \{s \mid s_0 \longrightarrow_T^* s\}$.

Definition 21 (LTS Language of Athena)**.** *The LTS uses first-order variables plus functional arrays/tuples and bit-ops (Lemma 8). Control-flow uses labeled locations and* `assume` *guards.*

Assumption 9 (Floating-Point Out of Scope)**.** *If floating-point operations occur in analyzed executions,* Athena *returns* Unknown*. All soundness statements assume they do not occur.*

5.3 Supported LLVM Subset and Totality of the Translation

Definition 22 (Supported LLVM Subset)**.** *Let $\mathcal{I}$ be the set of LLVM instruction forms handled by the LLVM→LTS translation (arithmetic, comparisons,* `br`*,* `phi`*, and array/tuple-based aggregate operations, plus calls to the inserted helper primitives). We say an LLVM program is* supported *if every executed instruction is in $\mathcal{I}$.*

Assumption 10 (Compiler Emits Supported LLVM on In-Scope Inputs)**.** *For in-scope inputs and the chosen mode, every LLVM execution of $\mathsf{Comp}(P_A)$ executes only instructions in $\mathcal{I}$ (Definition 22).*

Definition 23 (Instruction Encoding)**.** *For each $I \in \mathcal{I}$, define an LTS fragment $[\![I]\!]$. The full translation $T = \mathsf{LTS}(P_A)$ is obtained by composing these encodings over the LLVM CFG.*

Definition 24 (LLVM–LTS Relation $\mathcal{R}_{AL}$)**.** *$\mathcal{R}_{AL}(\Sigma, s)$ holds when SSA registers correspond to LTS variables, aggregate memory is represented by functional arrays/tuples, and the LLVM PC equals the LTS control label.*

Lemma 10 (Local Simulation Coverage)**.** *For every instruction form $I \in \mathcal{I}$, the corresponding encoding $[\![I]\!]$ is covered by exactly one of the local simulation lemmas below (arithmetic/icmp, branch/assume, phi, memory via arrays/tuples, helper calls).*

Proof. By inspection of the finite instruction-form grammar defining $\mathcal{I}$. □

Lemma 11 (Local Simulation: Arithmetic and Comparison)**.** *If LLVM executes $I \in \mathcal{I}$ of arithmetic/icmp form and $\mathcal{R}_{AL}(\Sigma, s)$ holds at its PC, then executing $[\![I]\!]$ yields s' such that $\mathcal{R}_{AL}(\Sigma', s')$ holds for the resulting LLVM state Σ'.*

Proof. Unfold the LLVM semantics of I and the LTS assignment/bit-op semantics. BV uses Lemma 8; Modulo uses Lemma 9. □

Lemma 12 (Local Simulation: Branch and Assume)**.** *If LLVM executes a branch and $\mathcal{R}_{AL}(\Sigma, s)$ holds, then the LTS encoding using* `assume` *reaches the corresponding successor label and preserves $\mathcal{R}_{AL}$.*

Proof. Case split on the condition value; `assume` blocks the infeasible edge. □

Lemma 13 (Local Simulation: Phi Nodes)**.** *At a join block, executing the LTS encoding of ϕ nodes assigns each LTS variable the value from the chosen incoming edge, matching LLVM ϕ semantics and preserving $\mathcal{R}_{AL}$.*

Proof. By definition of ϕ semantics and edge-labeled assignments in the encoding. □

Lemma 14 (Local Simulation: Aggregate/Memory via Arrays and Tuples)**.** *For aggregate operations represented using arrays/tuples, the LTS encoding with functional* `select/store` *and tuple constructors/selectors implements the same update as LLVM and preserves* $\mathcal{R}_{AL}$*.*

Proof. By definitional semantics of functional update and the compiler's lowering into explicit select/store forms. □

Theorem 2 (LLVM ⇒ LTS Step Simulation on Supported Code)**.** *Assume supported code (Assumption 10) and no floating point (Assumption 9). If* $\mathcal{R}_{AL}(\Sigma, s)$ *and* $\Sigma \longrightarrow_{\text{LLVM}} \Sigma'$ *executes one instruction, then there exists* s' *such that* $s \longrightarrow_T^* s'$ *and* $\mathcal{R}_{AL}(\Sigma', s')$*.*

Proof. By Lemma 10 and case analysis on the executed instruction form, applying the corresponding local lemma. □

Theorem 3 (LTS ⇒ LLVM Step Realization on Supported Code)**.** *Assume supported code. If* $\mathcal{R}_{AL}(\Sigma, s)$ *and* $s \longrightarrow_T s'$ *is a single LTS step, then there exists* Σ' *such that* $\Sigma \longrightarrow_{\text{LLVM}}^* \Sigma'$ *and* $\mathcal{R}_{AL}(\Sigma', s')$*.*

Proof. Each LTS step is part of some $[\![I]\!]$ for a concrete instruction/edge. Realize it by the corresponding LLVM instruction step(s). Boundedness follows because each $[\![I]\!]$ is finite. □

Theorem 4 (Infinite-Run Equivalence (LLVM ↔ LTS))**.** *Assume supported code and no floating point. Then* $\mathsf{Comp}(P_A)$ *has an infinite execution iff* $T = \mathsf{LTS}(P_A)$ *has an infinite execution.*

Proof. Use Theorems 2 and 3. Finite bounded stuttering preserves infiniteness. □

6 Certificate Checking Soundness

Assumption 11 (SMT Solver Soundness)**.** *All SMT validity checks used by* Athena *are sound: if the solver returns* valid*, the queried formula is valid in the intended background theories.*

Definition 25 (Inductive Invariant)**.** *A predicate* $\mathsf{Inv} : S \to \{\mathsf{true}, \mathsf{false}\}$ *is inductive for* $T = (S, s_0, \to)$ *iff: (i)* $\mathsf{Inv}(s_0)$ *and (ii)* $\forall s, s'.\ \mathsf{Inv}(s) \wedge (s \longrightarrow_T s') \Rightarrow \mathsf{Inv}(s')$*.*

Definition 26 (Invariant Checking Procedure)**.** Athena *computes a candidate* Inv *and performs SMT checks:* $\mathsf{Inv}(s_0)$ *and* $\forall s, s'.\ \mathsf{Inv}(s) \wedge (s \longrightarrow_T s') \Rightarrow \mathsf{Inv}(s')$*.*

Lemma 15 (Invariant Soundness from SMT Checks)**.** *If the SMT checks succeed, then* Inv *is inductive and hence* $\forall s \in \mathsf{Reach}(T).\ \mathsf{Inv}(s)$*.*

Proof. By Assumption 11, both checked formulas are valid, so Inv is inductive. Induction on path length yields reachability soundness. □

Definition 27 (Ranking Certificate and Validation)**.** *A ranking certificate is* $\rho : S \to (W, \prec)$ *with* $(W, \prec)$ *well-founded. Define* $\mathsf{ValidRank}(T, \rho)$ *iff for all* $s \longrightarrow_T s'$*,* $\mathsf{Inv}(s) \Rightarrow \rho(s') \prec \rho(s)$*.*

Definition 28 (Recurrent Certificate and Validation)**.** *A recurrent certificate is a non-empty* $S' \subseteq S$*. Define* $\mathsf{ValidRec}(T, S')$ *iff: (i)* $\exists s \in S' \cap \mathsf{Reach}(T)$*, (ii)* $S' \subseteq \{s \mid \mathsf{Inv}(s)\}$*, and (iii)* $\forall s \in S'.\ \exists s' \in S'.\ s \longrightarrow_T s'$*.*

Lemma 16 (Validator Soundness for Ranking Certificates)**.** *If* Athena *accepts a ranking certificate* ρ*, then* $\mathsf{ValidRank}(T, \rho)$ *holds.*

Proof. Acceptance means the decrease condition under Inv was SMT-checked and returned *valid*. By Assumption 11, the decrease property holds. □

Lemma 17 (Validator Soundness for Recurrent Certificates)**.** *If* ATHENA *accepts a recurrent certificate S', then* $\mathsf{ValidRec}(T, S')$ *holds.*

Proof. Acceptance means SMT-checked non-emptiness, existence of a reachable witness, $S' \Rightarrow \mathsf{Inv}$, and successor-closure. By Assumption 11, these properties hold in T. □

Lemma 18 (Ranking Soundness)**.** *If* $\mathsf{ValidRank}(T, \rho)$ *holds and* $\forall s \in \mathsf{Reach}(T).\mathsf{Inv}(s)$*, then T has no infinite execution.*

Proof. An infinite execution would induce an infinite strictly descending chain in $(W, \prec)$, contradiction. □

Lemma 19 (Recurrent-Set Soundness)**.** *If* $\mathsf{ValidRec}(T, S')$ *holds, then T has an infinite execution.*

Proof. From a reachable witness in S', repeatedly pick a successor in S' to build an infinite path. □

7 End-to-End Soundness

Theorem 5 (End-to-End Conditional Soundness of ATHENA)**.** *Let P be a C program satisfying Assumptions 1, 2, 4, 5, 6, 7, 9, 8, 10, 11, and 3. Let* $P_A = \mathsf{Rewrite}(P)$ *and let* $T = \mathsf{LTS}(P_A)$*. Assume* ATHENA*'s invariant SMT checks succeed and a certificate is accepted. Then:*

- *If* ATHENA *returns* TERMINATING*, then all executions of P are finite.*
- *If* ATHENA *returns* NONTERMINATING*, then P has an infinite execution.*
- *If* ATHENA *returns* UNKNOWN*, no claim is made.*

Proof. By Lemma 15, the checked invariant satisfies $\forall s \in \mathsf{Reach}(T).\mathsf{Inv}(s)$.

(TERMINATING) Acceptance of a ranking certificate implies $\mathsf{ValidRank}(T, \rho)$ (Lemma 16), hence T has no infinite execution (Lemma 18). By Theorem 4 and Assumption 8, P_A has no infinite execution. By Theorem 1(2), P has no infinite execution.

(NONTERMINATING) Acceptance of a recurrent certificate implies $\mathsf{ValidRec}(T, S')$ (Lemma 17), hence T has an infinite execution (Lemma 19). By Theorem 4 and Assumption 8, P_A has an infinite execution. By Theorem 1(1), P has an infinite execution.

(UNKNOWN) No claim. □